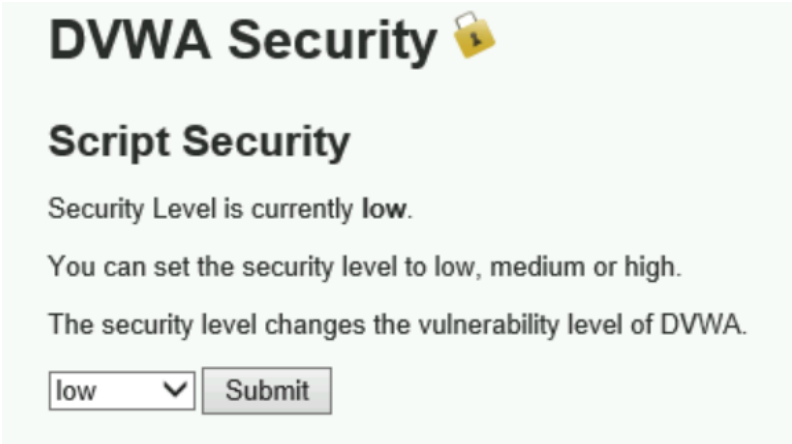
Lab 5: Attacking a Vulnerable Web Application and Database

This lab took us through the steps of how to know if a web application and database is vulnerable as well as how to exploit a vulnerability and attack the system. We used the XSS Reflected tab to figure out if there was a vulnerability and if the system allowed us to run scripts. We also used SQL injection attacks to discover information about the database and the User ID table, copying information into a tmp file, and retrieving the information from that tmp file. We also changed the DVWA security level between low and high to show us how the system is affected.

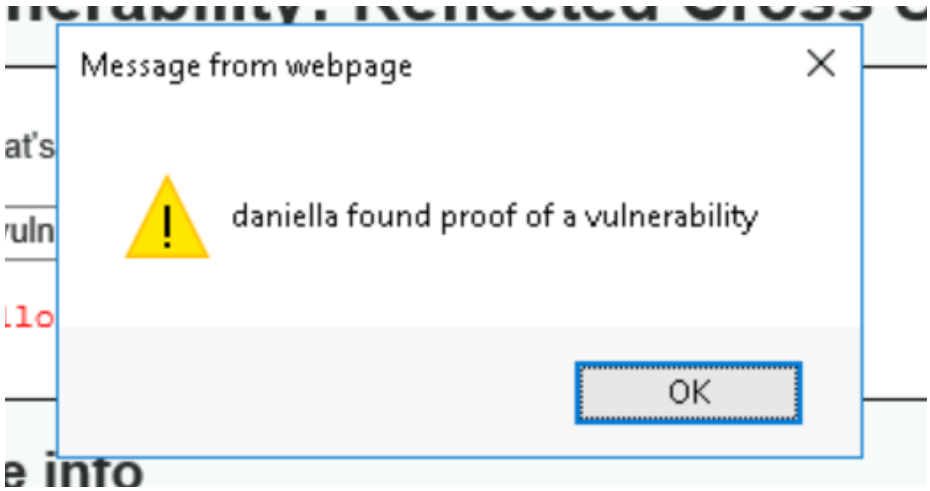
SECTION 1

Part 1: Connect to the DVWA

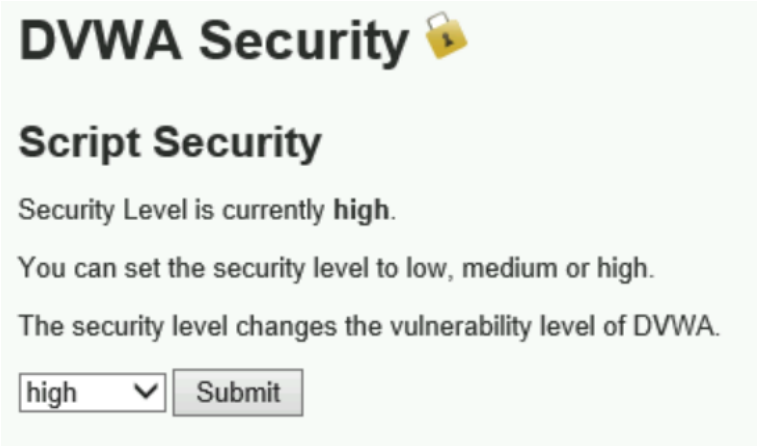


Screenshot 1: Change the security level from high to low

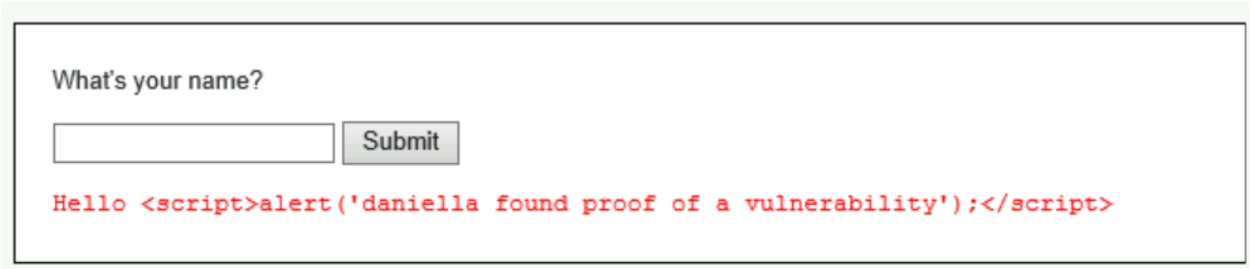
Part 2: Conduct an XSS Attack



Screenshot 2: Proof of the exposed XSS Reflected vulnerability

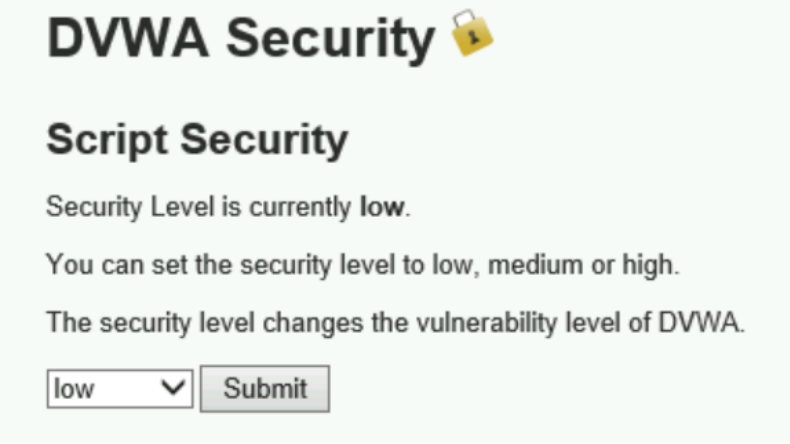


Screenshot 3: Change DVWA security level from low to high



Screenshot 4: Proof of script not running

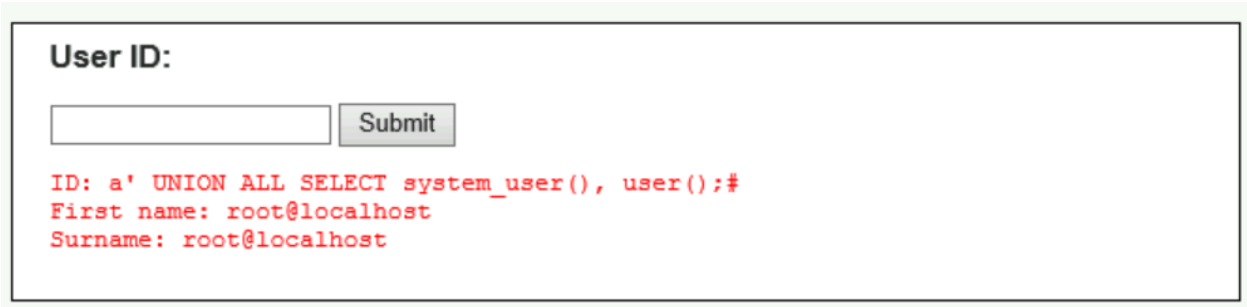
After changing the DVWA security level from low to high, the security of the system became secure and mitigated vulnerabilities. When the security level was low, we ran a script that generated a pop-up window stating that we found a vulnerability. However, once the security level was set to high, we ran the same script but instead of generating a pop-up window and running the script, the system returned our script.



Screenshot 5: Change DVWA security level from high back to low

Part 3: Conduct a SQL Injection Attack

In steps 7 through 9 we were asked to do a SQL injection test using the query, *a' ORDER BY 1;#* to determine how many columns there are in the USER ID table. For steps 7 and 8, after inputting and submitting the query, no error occurred meaning it accepted the query and there are at least 2 columns in the table. In step 9 we inputted the query *a' ORDER BY 3;#* and got an error saying "unknown column '3' in 'order clause'". This error syntax tells us that there is not a third column in the table.



Screenshot 6: User account information

```
ID: a' UNION ALL SELECT user, password FROM mysql.user;# priv;#
First name: root
Surname: *9CFBBC772F3F6C106020035386DA5BBBF1249A11

ID: a' UNION ALL SELECT user, password FROM mysql.user;# priv;#
First name: root
Surname: *9CFBBC772F3F6C106020035386DA5BBBF1249A11

ID: a' UNION ALL SELECT user, password FROM mysql.user;# priv;#
First name: root
Surname: *9CFBBC772F3F6C106020035386DA5BBBF1249A11

ID: a' UNION ALL SELECT user, password FROM mysql.user;# priv;#
First name: root
Surname: *9CFBBC772F3F6C106020035386DA5BBBF1249A11

ID: a' UNION ALL SELECT user, password FROM mysql.user;# priv;#
First name: debian-sys-maint
Surname: *75FAB0E9A569DBCA478523373F51B4D5D4CD664E

ID: a' UNION ALL SELECT user, password FROM mysql.user;# priv;#
First name: phpmyadmin
Surname: *9CFBBC772F3F6C106020035386DA5BBBF1249A11

ID: a' UNION ALL SELECT user, password FROM mysql.user;# priv;#
First name: tbaywolf
Surname: P@ssw0rdt
```

Screenshot 7: Hash for the user to the backend database

The main purpose of hashing in a database is to provide data integrity. Hashing is also a faster, more efficient way to process large amounts of information as well as a one-way protection from attackers.

Part 4: Retrieve the *yourname*_S1.txt file

```
root@dvwa:~# cat /var/lib/mysql/dvwa/daniella_S1.txt
test    123
root@dvwa:~#
```

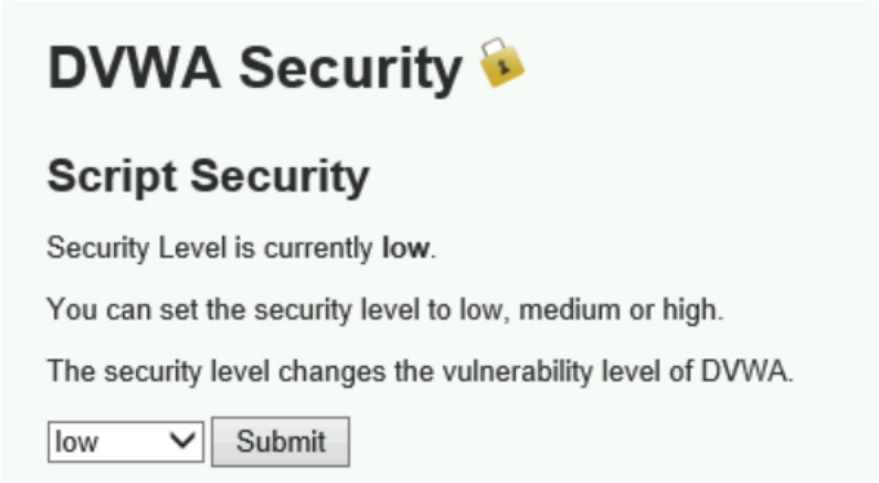
Screenshot 8: Contents of the *yourname*_S1.txt file

Briefly describe the security countermeasures you would recommend to mitigate the risk from compromise and exploitation.

One way to prevent XSS and SQL injection attacks is to sanitize any user input that is obtained from input fields. Ensuring that only valid characters and formats are accepted by the input field prevents user input from being interpreted as code. Users should not be able to run queries or copy data to a file if the input is sanitized properly. Implementing the least privilege principle may also help protect a database. Allowing only those whose roles require access to sensitive data in the database to be granted access would prevent an unauthorized user from running queries and gaining access to sensitive data.

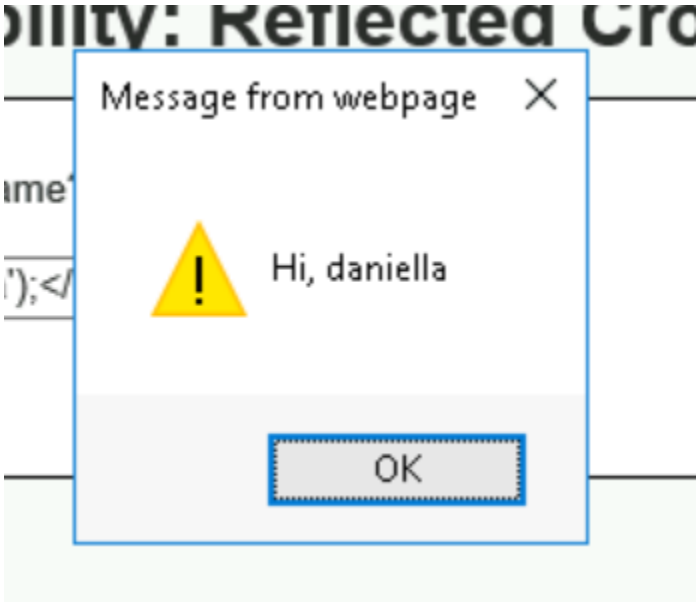
SECTION 2

Part 1: Connect to the DVWA



Screenshot 9: Setting the DVWA security level to low

Part 2: Conduct an XSS Attack

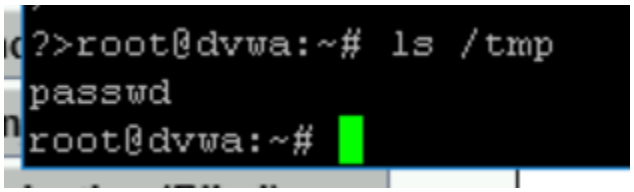


Screenshot 10: Proof of exposed XSS vulnerability

Screenshot 11: Proof of script not running when DVWA security level is set to high

When the security level is set to low, it is a vulnerable system allowing scripts to be run. We know that the system allows scripts to be run because when we typed *<script>alert('Hi, daniella');</script>* into the

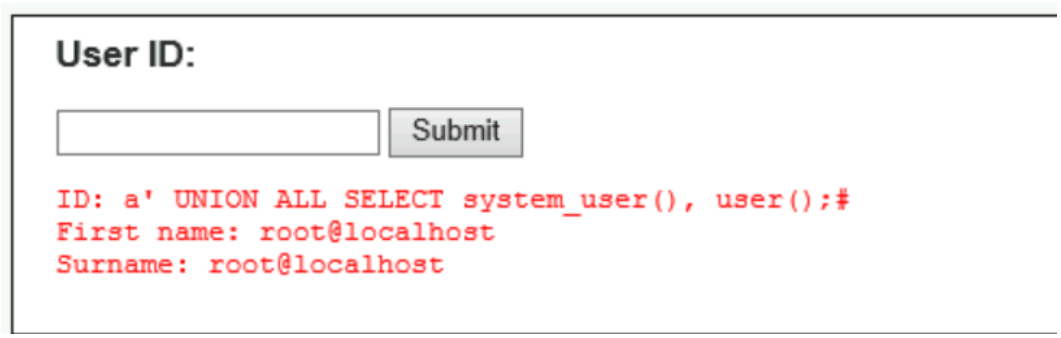
What's your name? box a pop-up window showed up telling us that it is allowing scripts to be run. However after we set the security level to high, it no longer allowed scripts to be run. We know this because when we typed *<script>alert('Hi, daniella');</script>* all the system did was return our script back to us rather than a pop-up window showing up so now we know the system is mitigating vulnerabilities.



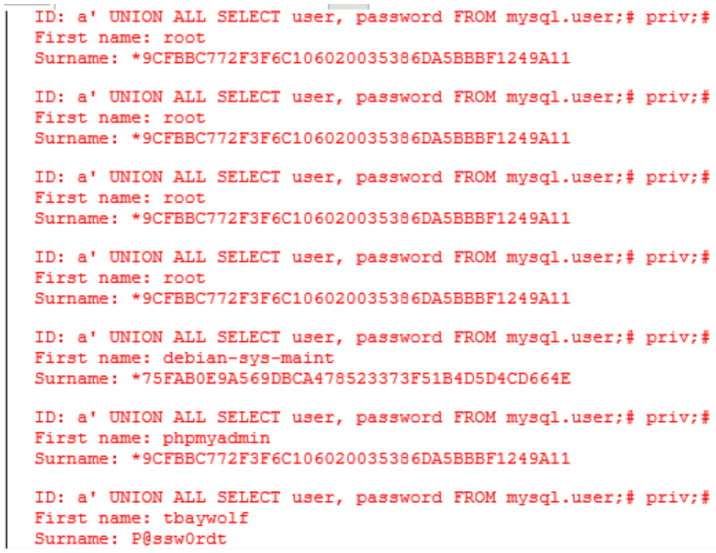
Screenshot 12: Proof that the passwd file was copied to tmp

Part 3: Conduct a SQL injection attack

In steps 4 through 6 we inserted SQL queries into the User ID box to attempt to find out how many columns there are in the User ID table. For steps 4 and 5 we received no error meaning that there are at least 2 columns in the table. Although, during step 6 we ran the query *a' ORDER BY 3;#* to see if there is a third column, but instead we got an error saying "Unknown column '3' in 'order clause'" meaning that there is not a third column in the table.



Screenshot 13: User information being used



Screenshot 14: Returned hashes for the user to the backend database

The main purpose of hashing in a database is to provide data integrity. Hashing is also a faster, more efficient way to process large amounts of information as well as a one-way protection from attackers.

Part 4: Retrieve the yourname_S2.txt file

```
root@dvwa:~# cat /var/lib/mysql/dvwa/daniella_S2.txt
successful      hack
root@dvwa:~#
```

Screenshot 15: Contents of the *yourname*_S2.txt file

Briefly describe the security countermeasures you would recommend to mitigate the risk from compromise and exploitation.

One way to prevent XSS and SQL injection attacks is to sanitize any user input that is obtained from input fields. Ensuring that only valid characters and formats are accepted by the input field prevents user input from being interpreted as code. Users should not be able to run queries or copy data to a file if the input is sanitized properly. Implementing the least privilege principle may also help protect a database. Allowing only those whose roles require access to sensitive data in the database to be granted access would prevent an unauthorized user from running queries and gaining access to sensitive data.